VERİ BİLİMCİ MÜFREDATI 2026

# 02

# Python Temelleri

Veri bilimi için sağlam Python temeli: syntax, veri yapıları, kontrol akışı, fonksiyonlar, modüller, hatalar ve OOP

**Bu modülün amacı**
Python sözdizimini ezberletmek değil; veriyi güvenli, okunabilir ve tekrar kullanılabilir şekilde işleyebilecek bir programlama düşüncesi kazandırmaktır. Modülün sonunda küçük analiz scriptleri yazabilecek, fonksiyonlaştırabilecek, hataları yönetebilecek ve Pandas/NumPy aşamasına hazır hale geleceksin.

**KONU ANLATIMI + KOD ÖRNEKLERİ + VERİ BİLİMİ VAKALARI + ALIŞTIRMALAR + MÜLAKAT SORULARI**

# 1. Bu modülde ne öğreneceksin?

- Python kodunun nasıl çalıştığını, indentation ve statement/expression farkını anlamak.
- int, float, bool, str ve None gibi temel veri tiplerini doğru kullanmak.
- list, tuple, set ve dict veri yapılarını problem tipine göre seçmek.
- if/elif/else, for ve while ile program akışını kontrol etmek.
- Fonksiyon tasarlamak; parametre, return, scope, *args ve **kwargs mantığını kavramak.
- List/dict/set comprehension ile okunabilir veri dönüşümleri yapmak.
- Modül, paket, pip ve virtual environment mantığını anlamak.
- Exception handling, dosya işlemleri ve context manager kullanmak.
- Veri bilimi için gerekli seviyede class, object ve method kavramlarını öğrenmek.
- Pythonic, test edilebilir ve bakımı kolay kod yazma alışkanlığı edinmek.

| Bölüm | Odak |
|---|---|
| 2-5 | Çalışma ortamı, syntax, değişkenler ve temel veri tipleri |
| 6-8 | String ve koleksiyon veri yapıları |
| 9-12 | Koşullar, döngüler, fonksiyonlar ve comprehensions |
| 13-16 | Modüller, exceptions, dosya işlemleri ve OOP |
| 17-19 | Iterator/generator, Pythonic kod, debugging/testing |
| 20-22 | Veri bilimi mini projesi, alıştırmalar ve mülakat soruları |

**Temel zihniyet**
Python öğrenirken hedef "syntax biliyorum" demek değil, bir problemi küçük adımlara ayırıp veriyi dönüştüren güvenilir kod yazabilmektir. Veri biliminde asıl hız, doğru abstraction ve tekrar kullanılabilir fonksiyonlardan gelir.

# 2. Python nasıl çalışır?

Python yüksek seviyeli, dinamik tipli ve yorumlanan bir programlama dilidir. Kaynak kodun CPython gibi bir implementation tarafından bytecode'a çevrilip Python virtual machine üzerinde çalıştırılması yaygın modeldir. Veri bilimi ekosisteminin büyük bölümü Python API'leriyle kullanılır; ağır sayısal işlemler çoğu zaman C/C++/Fortran tabanlı kütüphanelerde gerçekleşir.

| Kavram | Ne anlama gelir? | Pratik etkisi |
|---|---|---|
| Dinamik tip | Değişkenin tipi önceden deklarasyonla sabitlenmez | Hızlı geliştirme; ama tip hatalarını çalışma anında görebilirsin |
| Yorumlanan | Kod tipik olarak doğrudan runtime tarafından yürütülür | REPL ve notebook deneyimi hızlıdır |
| Garbage collection | Kullanılmayan nesnelerin belleği otomatik yönetilir | Manuel memory management ihtiyacı azalır |
| Batteries included | Standart kütüphane geniştir | Dosya, tarih, JSON, regex gibi işler için temel araçlar hazırdır |

## 2.1 Python nerede çalıştırılır?

- Terminal/REPL: hızlı denemeler.
- Jupyter Notebook/Lab: keşifsel analiz ve öğretim.
- .py script: tekrar kullanılabilir analiz ve otomasyon.
- IDE (VS Code/PyCharm): daha büyük projeler, debugging ve test.

Terminal örneği

```
python --version
python analysis.py
python -m pip --version
```

**Veri bilimci için öneri**
Notebook keşif için mükemmeldir; üretilebilir mantığı zamanla .py modüllerine taşımak iyi alışkanlıktır. Notebook hücrelerini rastgele sırada çalıştırmaya bağımlı bir proje reproducible değildir.

## 3. Syntax, indentation ve yorumlar

Python blokları süslü parantez yerine indentation ile tanımlar. Bu yüzden girinti yalnızca estetik değildir; programın anlamının bir parçasıdır. PEP 8 yaygın stil rehberidir ve genellikle 4 boşluk girinti kullanılır.

```
score = 82

if score >= 70:
    status = "passed"
    print(status)
else:
    print("failed")
```

| Yapı | Örnek | Not |
|---|---|---|
| Yorum | # churn oranını hesapla | Kodun nedenini açıklamak için; gereksiz "ne yaptığı" yorumu yazma |
| Satır devamı | Parantez içinde çok satır | Backslash yerine parantez tercih edilir |
| İsimlendirme | customer_count | Değişken/fonksiyonlarda snake_case |
| Sabit | MAX_RETRY = 3 | Konvansiyon olarak UPPER_CASE |
| Boolean | is_active = True | is_, has_, can_ gibi isimler okunabilirliği artırır |

**Sık hata**
Tab ve space karıştırmak veya yanlış girinti yapmak IndentationError üretebilir. Editöründe "insert spaces" ve otomatik formatter kullanmak bu sınıf hataları azaltır.

## 4. Değişkenler, nesneler ve atama

Python'da değişkeni bir kutu gibi değil, bir nesneye bağlanan isim gibi düşünmek daha doğrudur. Atama işlemi nesneyi kopyalamak zorunda değildir; aynı nesneye ikinci bir isim bağlayabilir.

```
x = 10
name = "Semih"
rate = 0.083
is_valid = True
missing_value = None
```

| Tip | Örnek | Veri bilimi kullanımı |
|---|---|---|

| | | |
|---|---|---|
| int | 42 | Sayaç, adet, id (matematiksel işlem yoksa id çoğu zaman string de olabilir) |
| float | 3.14 | Oran, ölçüm, model çıktısı |
| bool | True / False | Filtre, koşul, bayrak |
| str | "mobile" | Kategori, metin, id |
| None | None | Değer yok/atanmamış durumu |

### 4.1 Mutability neden önemli?

```
a = [1, 2]
b = a
b.append(3)

print(a)  # [1, 2, 3]
```

list mutable olduğu için a ve b aynı liste nesnesine işaret eder. Bağımsız kopya gerektiğinde copy işlemi düşünülmelidir. Bu davranış fonksiyonlarda ve nested yapılarda beklenmedik yan etkilere yol açabilir.

## 5. Operatörler ve tip dönüşümleri

| Kategori | Operatörler | Örnek |
|---|---|---|
| Aritmetik | + - * / // % ** | revenue / users |
| Karşılaştırma | == != > >= < <= | score >= 0.8 |
| Mantıksal | and, or, not | is_active and paid |
| Üyelik | in, not in | country in allowed |
| Identity | is, is not | value is None |

```
orders = 17
users = 5

print(orders / users)   # 3.4
print(orders // users)  # 3
print(orders % users)   # 2
print(2 ** 3)           # 8
```

### 5.1 Tip dönüşümü

```
raw_age = "30"
age = int(raw_age)
ratio = float("0.73")
label = str(2026)

bool(0)        # False
bool(1)        # True
bool("")       # False
bool("False")  # True: boş olmayan string
```

**Mülakat detayı**
None kontrolünde genellikle value == None yerine value is None kullanılır. "is" nesne kimliğini, "==" değer eşitliğini test eder.

# 6. String işlemleri

String immutable bir veri tipidir. Temizleme, kategorik değer standardizasyonu, log parsing ve basit NLP ön işlemlerinde yoğun kullanılır.

```
text = "  Data Science  "
clean = text.strip().lower()

print(clean)                # data science
print(clean.replace(" ", "_"))
print(clean.split())
print("science" in clean)
print(clean[0:4])           # data
```

| Method | Amaç | Örnek |
|---|---|---|
| strip() | Baş/son boşlukları temizler | " x ".strip() -> "x" |
| lower()/upper() | Harf standardizasyonu | "TR".lower() |
| split() | Stringi parçalara ayırır | "a,b".split(",") |
| join() | Parçaları birleştirir | ",".join(items) |
| startswith() | Prefix kontrolü | url.startswith("https") |
| replace() | Metin değişimi | text.replace("N/A", "") |

## 6.1 f-string

```
model = "logistic_regression"
auc = 0.8734
print(f"{model} AUC = {auc:.3f}")
# logistic_regression AUC = 0.873
```

# 7. List ve tuple

List sıralı ve mutable koleksiyondur. Tuple sıralı fakat immutable'dır. Veri bilimi kodunda feature listeleri, metrik isimleri, ara sonuçlar ve koordinat/konfigürasyon benzeri sabit gruplar için kullanılır.

```
features = ["age", "income", "tenure"]
features.append("orders")
features.remove("income")

first = features[0]
last_two = features[-2:]

shape = (1000, 12)
rows, cols = shape
```

| İşlem | List | Tuple |
|---|---|---|
| Sıralı | Evet | Evet |
| Mutable | Evet | Hayır |
| Indexing/slicing | Evet | Evet |
| Tipik kullanım | Değişen koleksiyon | Sabit kayıt / unpacking |

**Sık hata**
Bir fonksiyonun default parametresine [] gibi mutable nesne koymak, çağrılar arasında state paylaşılmasına yol açabilir. Default olarak None kullanıp fonksiyon içinde liste oluşturmak daha güvenlidir.

```
def add_feature(feature, features=None):
    if features is None:
        features = []
    features.append(feature)
    return features
```

# 8. Dictionary ve set

Dictionary key-value eşlemesi yapar. Set ise benzersiz elemanlardan oluşur ve hızlı üyelik kontrolü için kullanışlıdır.

```
customer = {
    "id": 101,
    "country": "TR",
    "orders": 7,
}

customer["orders"] += 1
segment = customer.get("segment", "unknown")

valid_countries = {"TR", "DE", "UK"}
print("TR" in valid_countries)
```

| Yapı | Güçlü olduğu iş | Örnek |
|---|---|---|
| dict | Anahtarla değer erişimi, JSON benzeri kayıt | metric -> value, config, API response |
| set | Benzersizlik, üyelik, küme işlemleri | unique ids, intersection, difference |

```
train_users = {1, 2, 3, 4}
test_users = {4, 5, 6}

overlap = train_users & test_users  # {4}
# Böyle bir overlap split leakage sinyali olabilir.
```

# 9. Koşullu ifadeler: if / elif / else

Koşullar, iş kurallarını ve veri kalitesi kararlarını kodda ifade eder. Önce özel ve kritik koşulları, sonra daha genel koşulları sıralamak okunabilirliği artırır.

```
churn_probability = 0.82

if churn_probability >= 0.80:
    risk = "high"
elif churn_probability >= 0.50:
    risk = "medium"
else:
    risk = "low"
```

```
country = "TR"
is_premium = True

if country == "TR" and is_premium:
    offer = "premium_tr_offer"
```

### 9.1 Truthy / falsy

| Falsy örnekler | Truthy örnekler |
|---|---|
| False, None, 0, 0.0, "", [], {}, set() | Çoğu diğer nesne; ör. 1, "text", [0] |

**Okunabilirlik**
if len(items) > 0 yerine çoğu durumda if items yazmak daha Pythonic'tir. Ancak veri yapısının truth-value davranışı belirsizse açık kontrol tercih edilir; NumPy/Pandas nesnelerinde bu konu özellikle önemlidir.

## 10. Döngüler: for ve while

Veri biliminde büyük tablolar üzerinde satır satır Python döngüsü çoğu zaman Pandas/NumPy vektörizasyonundan yavaştır. Yine de dosya listeleri, model konfigürasyonları, API sayfaları ve küçük koleksiyonlarda döngüler temel araçtır.

```
scores = [0.71, 0.83, 0.79]

for i, score in enumerate(scores, start=1):
    print(i, score)

metrics = {"auc": 0.84, "f1": 0.76}
for name, value in metrics.items():
    print(name, value)
```

```
attempt = 0
while attempt < 3:
    attempt += 1
    if attempt == 2:
        continue
    if attempt == 3:
        break
```

| Araç | Ne yapar? |
|---|---|
| range() | Sayı dizisi üretir |
| enumerate() | Index + değer verir |
| zip() | Birden çok iterable'ı paralel dolaştırır |

| | |
|---|---|
| break | Döngüyü sonlandırır |
| continue | Mevcut iterasyonu atlar |

# 11. Fonksiyonlar

Fonksiyonlar tekrar eden mantığı isimlendirir, test etmeyi kolaylaştırır ve notebook'taki tek seferlik kodu yeniden kullanılabilir bileşene dönüştürür. İyi fonksiyon tek bir net sorumluluğa sahip olmalıdır.

```
def conversion_rate(conversions, visitors):
    """Return conversion rate; None when denominator is zero."""
    if visitors == 0:
        return None
    return conversions / visitors

rate = conversion_rate(820, 10_000)
```

| Parça | Anlamı |
|---|---|
| def | Fonksiyon tanımı |
| Parametre | Fonksiyonun kabul ettiği isim |
| Argument | Çağrıda verilen gerçek değer |
| return | Fonksiyonun dışarı verdiği sonuç |
| docstring | Fonksiyonun sözleşmesini açıklayan metin |

## 11.1 Default ve keyword arguments

```
def summarize(values, decimals=2, prefix=""):
    mean_value = sum(values) / len(values)
    return f"{prefix}{mean_value:.{decimals}f}"

summarize([1, 2, 3], decimals=1, prefix="avg=")
```

**Fonksiyon sözleşmesi**
Bir fonksiyon için "hangi inputları kabul eder, ne döndürür, hangi durumda hata verir, yan etkisi var mı?" sorularını cevaplayabiliyorsan kod daha kolay test edilir ve yeniden kullanılır.

# 12. Scope, *args, **kwargs ve lambda

Scope bir ismin nerede erişilebilir olduğunu belirler. Genel olarak global state'i azaltıp değerleri parametrelerle geçirmek daha güvenli ve test edilebilir kod üretir.

```
threshold = 0.50  # global

def classify(score, threshold=0.50):
    label = int(score >= threshold)  # local
    return label
```

```
def mean(*values):
    return sum(values) / len(values)

mean(4, 8, 10)

config = {"threshold": 0.7}
# some_function(**config)  -> key'ler keyword argument olur
```

### 12.1 Lambda

```
rows = [
    {"name": "A", "revenue": 120},
    {"name": "B", "revenue": 90},
]

sorted_rows = sorted(rows, key=lambda row: row["revenue"], reverse=True)
```

**Ne zaman lambda?**
Kısa, tek ifadeli ve yerel dönüşümlerde kullanışlıdır. İş mantığı büyüyorsa isimli def fonksiyonu daha okunabilir, belgelenebilir ve test edilebilirdir.

## 13. Comprehensions

Comprehension yapıları küçük koleksiyon dönüşümlerini açık ve kompakt biçimde ifade eder. Aşırı iç içe koşullar okunabilirliği düşürür; o noktada normal döngü veya fonksiyon daha iyidir.

```
values = [2, 5, 8, 11]

squares = [x ** 2 for x in values]
even_squares = [x ** 2 for x in values if x % 2 == 0]

metric_names = ["AUC", "F1", "Recall"]
normalized = {name.lower(): None for name in metric_names}

unique_lengths = {len(name) for name in metric_names}
```

| Yapı | Örnek sonuç |
|---|---|
| List comprehension | Filtrelenmiş/dönüştürülmüş liste |
| Dict comprehension | Key-value mapping |
| Set comprehension | Benzersiz dönüşüm sonuçları |

## 14. Modüller, paketler, pip ve virtual environment

Bir .py dosyası modül olabilir; birden fazla modül paket halinde organize edilebilir. Veri bilimi projelerinde bağımlılıkları izole etmek reproducibility için kritik önemdedir.

```
# metrics.py
def rmse(y_true, y_pred):
    ...

# analysis.py
from metrics import rmse
```

**Ortam yönetimi**

```
python -m venv .venv

# Linux/macOS
source .venv/bin/activate

python -m pip install pandas scikit-learn
python -m pip freeze > requirements.txt
```

| Kavram | Amaç |
|---|---|
| module | Kodun mantıksal bir dosyada toplanması |
| package | Modüllerin dizin yapısında gruplanması |
| pip | Paket kurulum aracı |
| venv | Proje bağımlılıklarını izole etme |
| requirements.txt | Bağımlılık sürümlerini paylaşma |

**Reproducibility**
"Bende çalışıyor" veri biliminde yeterli değildir. Python sürümü, paket sürümleri, random seed ve giriş verisi tanımı mümkün olduğunca kaydedilmelidir.

## 15. Exception handling

Hata yönetimi, hatayı gizlemek değil; beklenen hata koşullarını açıkça ele almak ve beklenmeyen hataları görünür bırakmaktır. Çok geniş try/except blokları debugging'i zorlaştırır.

```
def parse_age(raw):
    try:
        age = int(raw)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"Invalid age: {raw!r}") from exc

    if age < 0:
        raise ValueError("Age cannot be negative")
    return age
```

| Anahtar kelime | Rol |
|---|---|
| try | Hata oluşabilecek kod |
| except | Belirli exception'ı yakalar |
| else | Hata olmadığında çalışır |
| finally | Her durumda çalışır |
| raise | Bilinçli olarak exception üretir |

**Anti-pattern**
except Exception: pass veri kalitesi problemlerini sessizce yok edebilir. Bunun yerine beklenen exception türünü yakala, context ekle ve gerektiğinde yeniden raise et.

## 16. Dosya işlemleri, context manager ve pathlib

Dosya okurken with kullanmak kaynağın güvenli biçimde kapanmasını sağlar. pathlib platformlar arası dosya yolu yönetimini sadeleştirir.

```
from pathlib import Path

path = Path("data") / "customers.txt"

with path.open("r", encoding="utf-8") as f:
    first_line = f.readline().strip()

output = Path("reports") / "summary.txt"
output.parent.mkdir(parents=True, exist_ok=True)
output.write_text("analysis complete", encoding="utf-8")
```

```
import json

config = {"threshold": 0.65, "model": "baseline"}
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, ensure_ascii=False, indent=2)
```

**Veri bilimi bağlantısı**
CSV/Parquet işlemlerini daha sonra Pandas ile yapacaksın; fakat dosya yolu, encoding, JSON config ve output klasörü yönetimi saf Python bilgisidir ve gerçek projelerde sürekli kullanılır.

## 17. OOP temelleri: class, object ve method

Veri bilimci için OOP'nin hedefi karmaşık class hiyerarşileri kurmak değil; state ve davranışı gerektiğinde tek bir anlaşılır nesnede toplamak, kütüphane API'lerini okuyabilmek ve sklearn benzeri tasarımları anlayabilmektir.

```
class Standardizer:
    def __init__(self):
        self.mean_ = None

    def fit(self, values):
        self.mean_ = sum(values) / len(values)
        return self

    def transform(self, values):
        if self.mean_ is None:
            raise RuntimeError("Call fit first")
        return [x - self.mean_ for x in values]

transformer = Standardizer().fit([10, 12, 14])
print(transformer.transform([11, 15]))
```

| Kavram | Anlam |
| --- | --- |
| class | Nesnelerin şablonu |

| | |
|---|---|
| object/instance | Class'tan oluşturulan somut nesne |
| __init__ | Başlangıç state'ini kurar |
| self | O instance'a referans |
| method | Instance davranışı |
| attribute | Instance üzerinde tutulan veri/state |

### 17.1 Composition çoğu zaman inheritance'dan önce

Bir veri bilimi projesinde küçük fonksiyonlar ve birleştirilebilir nesneler çoğu durumda derin inheritance zincirlerinden daha basit ve sürdürülebilirdir. OOP bir hedef değil, karmaşıklığı yönetme aracıdır.

## 18. Iterable, iterator ve generator

Büyük veriyi belleğe tamamen almak yerine parça parça işlemek gerektiğinde iterator/generator yaklaşımı önem kazanır. Generator, yield ile değerleri ihtiyaç oldukça üretir.

```
def batches(items, batch_size):
    for start in range(0, len(items), batch_size):
        yield items[start:start + batch_size]

for batch in batches(list(range(10)), 3):
    print(batch)
```

| Terim | Kısa tanım |
|---|---|
| Iterable | for ile dolaşılabilen nesne |
| Iterator | next() ile sıradaki değeri veren nesne |
| Generator | yield ile lazy değer üreten iterator türü |

**Neden önemli?**
Milyonlarca kayıt, log dosyaları veya streaming kaynaklarında tüm veriyi RAM'e almak gereksiz veya imkânsız olabilir. Generator mantığı batch processing düşüncesinin temel taşlarından biridir.

## 19. Pythonic kod, debugging ve test alışkanlığı

İyi Python kodu yalnızca çalışmaz; niyeti belli eder. İsimlendirme, küçük fonksiyonlar, erken validation ve basit testler veri analizindeki sessiz hataları ciddi ölçüde azaltır.

| Kötü alışkanlık | Daha iyi yaklaşım |
|---|---|
| x1, x2, tmp gibi belirsiz isimler | conversion_rate, customer_ids gibi anlamlı isimler |
| Tek fonksiyonda 100 satır | Tek sorumluluklu küçük fonksiyonlar |
| print ile rastgele debug | Debugger/logging/assert veya hedefli kontroller |
| Magic number | İsimlendirilmiş constant/config |

| | |
|---|---|
| Input varsayımı | Açık validation |
| Kopyala-yapıştır kod | Fonksiyon/modül ile tekrar kullanma |

```
def conversion_rate(conversions: int, visitors: int) -> float:
    if conversions < 0 or visitors <= 0:
        raise ValueError("Counts must be valid")
    if conversions > visitors:
        raise ValueError("Conversions cannot exceed visitors")
    return conversions / visitors

assert conversion_rate(8, 100) == 0.08
```

**Type hints**
Type hint çalışma zamanında otomatik güvenlik sağlamaz; ama IDE, linter, static checker ve okuyucu için fonksiyonun beklediği veri biçimini görünür kılar.

## 20. Mini proje: E-commerce sipariş özetleyici

Bu mini proje yalnızca saf Python kullanarak ham sipariş kayıtlarını kontrol eder, KPI'lar üretir ve segment bazlı özet çıkarır. Pandas'a geçmeden önce programlama mantığını oturtmak için idealdir.

```
orders = [
    {"order_id": 1, "country": "TR", "amount": 120.0, "refunded": False},
    {"order_id": 2, "country": "DE", "amount": 80.0,  "refunded": True},
    {"order_id": 3, "country": "TR", "amount": 200.0, "refunded": False},
    {"order_id": 4, "country": "UK", "amount": 150.0, "refunded": False},
]
```

1. Geçersiz veya negatif amount değerlerini validation ile yakala.
2. Net revenue'yu refunded olmayan siparişlerden hesapla.
3. Ülke bazında order_count ve revenue sözlüğü üret.
4. AOV = net revenue / valid orders hesapla.
5. En yüksek gelirli ülkeyi bul.
6. Kodunu calculate_kpis() ve summarize_by_country() fonksiyonlarına böl.
7. Sonuçları JSON veya text dosyasına yaz.

```
from collections import defaultdict

def summarize_by_country(orders):
    summary = defaultdict(lambda: {"orders": 0, "revenue": 0.0})

    for order in orders:
        if order["amount"] < 0:
            raise ValueError("Negative amount")
        if order["refunded"]:
            continue

        country = order["country"]
        summary[country]["orders"] += 1
        summary[country]["revenue"] += order["amount"]

    return dict(summary)
```

**Portföy notu**
Bu mini proje tek başına büyük bir CV projesi değildir. Ama GitHub'da "Python fundamentals" bölümü için küçük, testli ve temiz bir örnek olarak kullanılabilir. Daha sonra aynı problemi Pandas ile tekrar çözerek farkı gösterebilirsin.

# 21. Uygulama alıştırmaları

| Seviye | Görev |
|---|---|
| 1 | Bir sayı listesinden min, max, mean değerlerini built-in fonksiyonlarla hesapla. |
| 1 | Bir string listesini strip + lower ile normalize et; boş değerleri çıkar. |
| 1 | Bir ürün sözlüğünde stok 0 ise "out_of_stock", aksi halde "available" etiketi üret. |
| 2 | Transaction listesinde kullanıcı başına toplam harcamayı dict ile hesapla. |
| 2 | İki kullanıcı setinin kesişimini bul ve train-test overlap kontrolü yaz. |
| 2 | Bir fonksiyonla precision = TP/(TP+FP) hesapla; sıfır paydayı ele al. |
| 2 | CSV yerine basit text dosyasındaki satırları generator ile tek tek oku. |
| 3 | Config dict alan bir model-evaluation fonksiyonu yaz ve **kwargs kullan. |
| 3 | Basit MetricTracker class'ı tasarla: add(), mean(), reset() methodları olsun. |
| 3 | Mini projen için 5 adet assert/test case yaz; edge case'leri özellikle test et. |

## 21.1 Kendini test et

- List ve tuple arasındaki temel fark nedir?
- is ile == neden aynı şey değildir?
- Mutable default argument neden risklidir?
- Generator hangi problemde listeden daha uygundur?
- try/except ne zaman kullanılmalı, ne zaman hata yükseltilmelidir?
- Global değişkenlere aşırı bağımlılık neden test edilebilirliği düşürür?

# 22. Mülakat soruları ve kısa cevap çerçevesi

| Soru | Beklenen cevap yönü |
|---|---|
| List vs tuple? | List mutable; tuple immutable. Semantik olarak değişmeyecek kayıt için tuple tercih edilebilir. |
| dict lookup neden kullanışlıdır? | Key tabanlı erişim; ortalama durumda hızlı lookup. Mapping/config/aggregation için uygundur. |
| Shallow copy vs deep copy? | Shallow nested nesneleri paylaşabilir; deep copy iç içe nesneleri de kopyalar. |
| *args ve **kwargs? | Değişken sayıda positional ve keyword argument toplar/açar. |
| Generator avantajı? | Lazy evaluation ve daha düşük memory kullanımı; tek geçişli akışlar için iyi. |

| | |
|---|---|
| Exception'ı neden dar yakalarsın? | Beklenmeyen hatayı gizlememek ve doğru recovery davranışı sağlamak için. |
| Mutable default arg problemi? | Default nesne function definition anında bir kez oluşturulur; çağrılar state paylaşabilir. |
| Comprehension ne zaman kötüleşir? | Çoklu nested loop/koşul okunabilirliği bozduğunda normal döngü/fonksiyon daha iyi. |
| OOP veri biliminde nerede görülür? | Estimator API'leri, pipeline bileşenleri, stateful transformer/model wrapper'ları. |
| Notebook kodunu nasıl production'a yaklaştırırsın? | Fonksiyonlaştır, modüllere taşı, config/logging/test ekle, bağımlılıkları sabitle. |

# 23. Final checklist ve çalışma planı

| Kontrol | Başardım |
|---|---|
| Temel veri tiplerini ve truthy/falsy davranışını açıklayabiliyorum | ☐ |
| List/tuple/set/dict arasında bilinçli seçim yapabiliyorum | ☐ |
| if/for/while ile küçük algoritmalar yazabiliyorum | ☐ |
| Fonksiyon yazıp input validation ve return tasarlayabiliyorum | ☐ |
| Scope, mutable nesne ve copy davranışlarının temel risklerini biliyorum | ☐ |
| Comprehension, enumerate, zip ve dict.items() kullanabiliyorum | ☐ |
| venv + pip ile izole ortam kurabiliyorum | ☐ |
| Belirli exception türlerini doğru biçimde yönetebiliyorum | ☐ |
| with ve pathlib ile dosya işlemleri yapabiliyorum | ☐ |
| Basit class okuyup yazabiliyorum; sklearn tarzı fit/transform fikrini anlıyorum | ☐ |
| Generator'ın neden memory açısından yararlı olduğunu açıklayabiliyorum | ☐ |
| Küçük kodu fonksiyonlaştırıp test/assert yazabiliyorum | ☐ |

## 23.1 7 günlük pratik planı

| Gün | Çalışma |
|---|---|
| 1 | Tipler, operatörler, string - 20 küçük soru |
| 2 | List/tuple/dict/set - aggregation ve membership alıştırmaları |
| 3 | if/for/while - 15 algoritmik mini problem |

| | |
|---|---|
| 4 | Fonksiyonlar, scope, comprehensions - mevcut kodu refactor et |
| 5 | Modules, venv, files, JSON, exceptions |
| 6 | OOP, generators, debugging ve test |
| 7 | E-commerce mini projesini sıfırdan süre tutarak tekrar yaz |

**Modülü bitirme kriteri**
Syntax'a bakmadan her şeyi ezberlemek gerekmiyor. Asıl kriter: dokümantasyon yardımıyla küçük bir problemi güvenilir şekilde çözebilmek, kodu parçalara ayırabilmek ve hatanın nerede olduğunu sistematik biçimde bulabilmek. Bundan sonra NumPy ve Pandas çok daha anlamlı hale gelir.

# Özet

Python temeli; değişken, döngü ve fonksiyonları ayrı ayrı bilmekten daha fazlasıdır. Veri bilimci için güçlü Python; veriyi doğru yapıda temsil etmek, iş mantığını fonksiyonlara ayırmak, yan etkileri sınırlamak, hataları görünür kılmak ve aynı analizi tekrar çalıştırılabilir hale getirmektir. Bu temelin üzerine NumPy, Pandas, scikit-learn ve üretim araçları inşa edilir.